Guía de estudio — TP3, Ejercicio 3

Banco de medición. Sistemas de Tiempo Real

1. Los números que tenés que saber de memoria

Escala

Reloj del micro	16 MHz	1 unidad	4 ticks = **64 ms**
Tick de FreeRTOS	**16 ms** (Watchdog)	C de la tarea D	1 unidad = **64 ms**
`portTICK_PERIOD_MS`	$2^4 = 16$	T de la tarea D	3 unidades = **192 ms**
`configTICK_RATE_HZ`	$128000 \gg 11 = 62$ Hz	$U = C/T$	**1/3**

Medido: pulso **64,03 ms** (error 0,05 %), período **192,00 ms** (exacto), ciclo de trabajo 1/3, plazos perdidos 0.

En la terminal: `perT = 8` ticks de espera, `dtT = 0` ticks durante la ráfaga, activaciones **+24** por reporte (el reporte sale cada 192 ticks: $192 \div 8 = 24$).

Del lazo: `_delay_loop_2` consume exactamente 4 ciclos por iteración, así que `ITER_POR_MS` $= 16\,000\,000/1000/4 = 4000$.

2. La historia, en ocho pasos

Si tenés que contar qué hiciste, es esta secuencia:

1. El cómputo se simula con **espera activa**, no con una llamada de bloqueo, porque C es demanda de CPU y el bloqueo justamente la cede.
2. El lazo cuenta **trabajo ejecutado** (iteraciones), no tiempo de reloj, para que una expropiación no le regale cómputo a la tarea.
3. El tick real es de 16 ms y no de los 15 del enunciado, así que se expresó todo **en ticks** y 1 unidad quedó en 4 ticks = 64 ms.
4. Primera medición: pulso 64 ms correcto, pero período **256 ms** en lugar de 192. El cómputo estaba bien; el período no.
5. Se instrumentó el firmware para que reportara los ticks consumidos por la ráfaga. Resultado: **`dtT` = 0**.
6. Diagnóstico: el contador de ticks no avanza durante la espera activa, así que `xTaskDelayUntil` agregaba el período entero *después* del cómputo. Período efectivo $= C + T = 256$ ms.
7. Corrección: descontar de la espera los ticks de la ráfaga, $12 - 4 = 8$ ticks.
8. Verificación por dos vías: el analizador da 192,00 ms y las activaciones por reporte pasaron de 16 a 24, exactamente lo predicho.

3. Preguntas probables y respuesta corta

- **¿Por qué espera activa y no `vTaskDelay`?** Porque `vTaskDelay` cede el procesador: simula una tarea que espera, no una que computa. Con bloqueo el factor de utilización efectivo sería cero y nunca se perdería un plazo.
- **¿Qué simula cada alternativa?** La espera activa, cómputo. La llamada de bloqueo, latencia: E/S, temporizador, evento externo.
- **¿Por qué `_delay_loop_2` y no comparar `millis()`?** El lazo tiene que contar trabajo, no tiempo. Contra reloj de pared, el tiempo que la tarea estuvo desalojada se contaría como cómputo y C se achicaría apenas apareciera competencia.
- **¿Por qué el tick es de 16 ms y no de 15?** Lo genera el Watchdog Timer con `WDTO_15MS`, que son 2048 ciclos del oscilador interno de 128 kHz, o sea 16 ms exactos. No es configurable.
- **¿El sistema es planificable?** $U = 1/3$. La cota de Liu y Layland para $n = 1$ es $1(2^1 - 1) = 1$, y $1/3 < 1$. Además es una sola tarea, no hay con quién competir.
- **¿Por qué A0, A1 y A2 están planas?** Porque el inciso pide una única tarea activa. Solo D conmuta PB3; los otros tres son salidas en cero hasta el Ejercicio 4.
- **¿Qué garantiza jitter = 0?** `xTaskDelayUntil` es una espera *absoluta*: el próximo despertar se calcula sobre el anterior, no sobre el instante actual, así que no acumula deriva. Con `vTaskDelay` el error de cada ciclo se sumaría.
- **¿Por qué la tarea de reporte tiene menor prioridad?** Para que no interfiera con las periódicas. Es instrumentación, no carga de tiempo real, y solo debe correr con el tiempo que sobra.
- **¿Qué pasa si aumento C?** El período efectivo crece y, cuando C se acerca a T, empiezan a perderse plazos. Con $U > 1$ es seguro.

- **¿Por qué se colgaba el sistema?** El bloque de reporte superaba los 64 bytes del buffer de transmisión del USART. `Serial.print` quedaba esperando dentro del controlador y el sistema entero se detenía. Se resolvió acortando las líneas.

4. Las tres cosas que no hay que decir mal

Cuidado

- El tick **no** mide 15 ms. Mide 16. La escala del enunciado es nominal; la del banco es la real.
- El `Rmax` de la terminal tiene **resolución de un tick** y no ve lo que pasa dentro de una ráfaga (`dtT` $= 0$). La medición válida de C y de R es el ancho de pulso del analizador.
- En el Ejercicio 4 el ancho de pulso **ya no es** C: el pin queda alto mientras la tarea está expropiada, así que mide R. Para obtener C hay que sumar solo los tramos en que ningún otro pin esté alto.

5. Cómo mostrar que funciona, en un minuto

1. Abrir el proyecto y compilar. La terminal arranca con `tick=16ms unidad=64ms`: ahí ya queda fijada la escala.
2. Dejar correr hasta ver tres o cuatro líneas de reporte. Señalar que `activ` sube de a 24, que `perdidos` es 0 y que `perT` es 8.
3. En el Logic Analyser: *Capture Resolution* 0,1 ms, desmarcar y volver a marcar `Capture`, dejar que se llene.
4. Cursores de subida a bajada: **64,03 ms**. De subida a subida: **192,00 ms**. Cerrar diciendo que son los valores teóricos 4×16 y 12×16.